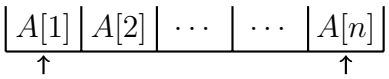
1) **Cálculo Linear**: O algoritmo abaixo percorre um vetor A de n elementos para somá-los. Preencha a tabela de custos:



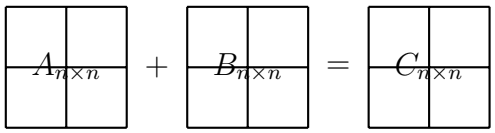
Percorre as posições sequencialmente

SOMA-VETOR(A, n)

	instrução	**custo**	**vezes**
1	$soma = 0$	c_1	1
2	**para** $i = 1$ **até** n:	c_2	$n + 1$
3	$\quad soma = soma + A[i]$	c_3	n
4	**retornar** $soma$	c_4	1

Resposta: Custo total: $T(n) = c_1 + c_2(n + 1) + c_3n + c_4$. O termo dominante é linear, portanto a complexidade é $\mathbf{O(n)}$.

2) **Cálculo Quadrático**: Duas matrizes A e B de ordem $n \times n$ são somadas. Analise o impacto dos laços aninhados:



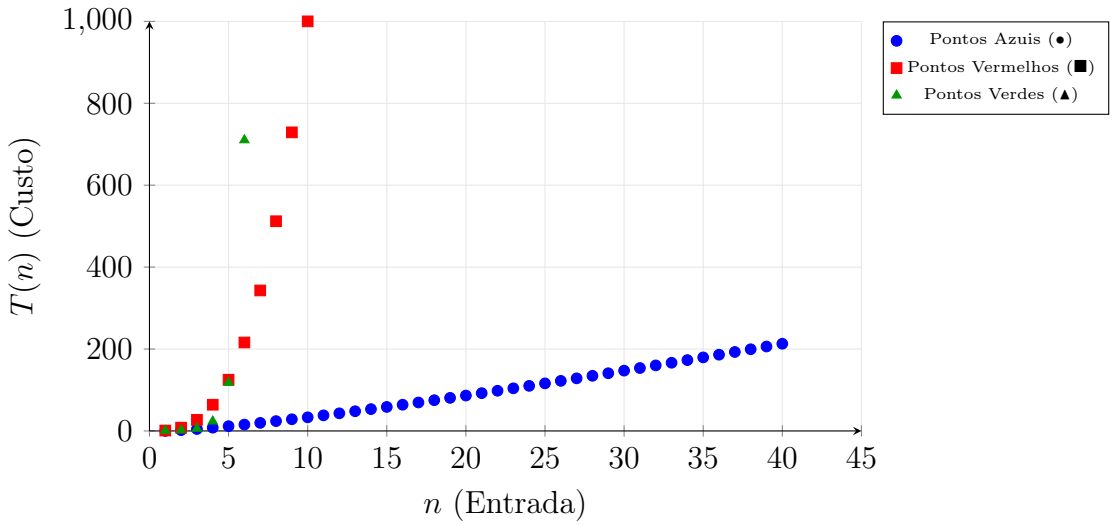
Cada célula (i, j) exige um cálculo

SOMA-MATRIZ(A, B, n)

	instrução	**custo**	**vezes**
1	**para** $i = 1$ **até** n:	c_1	$n + 1$
2	$\quad$ **para** $j = 1$ **até** n:	c_2	$n(n + 1)$
3	$\quad\quad C[i, j] = A[i, j] + B[i, j]$	c_3	n^2
4	**retornar** C	c_4	1

Resposta: Custo total: $T(n) = c_1(n + 1) + c_2(n^2 + n) + c_3n^2 + c_4$. O termo dominante é quadrático (n^2), portanto a complexidade é $\mathbf{O(n^2)}$.

3) **Análise Gráfica de Pontos**: Com base no comportamento das curvas no gráfico abaixo, assinale a alternativa que associa corretamente a cor/formato do ponto à sua respectiva complexidade:



a) Azul: $O(n^3)$; Vermelho: $O(n!)$; Verde: $O(n \log n)$.

b) Azul: $O(n!)$; Vermelho: $O(n^3)$; Verde: $O(n \log n)$.

c) Azul: $O(n \log n)$; Vermelho: $O(n^3)$; Verde: $O(n!)$.

c) Azul: $O(n \log n)$; Vermelho: $O(n!)$; Verde: $O(n^3)$.

4) **Busca em Matriz (Busca Exaustiva)**:

BUSCA-MATRIZ$(M, n, alvo)$

	instrução	**custo**	**vezes** (Pior Caso)
1	**para** $i = 1$ **até** n:	c_1	$n + 1$
2	**para** $j = 1$ **até** n:	c_2	$n(n + 1)$
3	**se** $M[i, j] == alvo$:	c_3	n^2
4	**retornar** (i, j)	c_4	0 ou 1
5	**retornar** NULO	c_5	0 ou 1

Resposta: No pior caso (o elemento não está na matriz ou está na última posição), o algoritmo percorrerá todos os n^2 elementos da matriz, resultando em uma complexidade de $\mathbf{O(n^2)}$.

5) **Laços Independentes vs Aninhados**: O algoritmo abaixo processa o vetor A de duas formas diferentes. Qual termo domina o custo final?

ALGORITMO-MISTO(A, n)

	instrução	custo	vezes
1	**para** $i = 1$ **até** n:	c_1	$n + 1$
2	print$(A[i])$	c_2	n
3	**para** $j = 1$ **até** n:	c_3	$n + 1$
4	**para** $k = 1$ **até** n:	c_4	$n(n + 1)$
5	print$(A[j] + A[k])$	c_5	n^2

Resposta: O laço das linhas 1 e 2 tem complexidade $O(n)$, enquanto os laços aninhados das linhas 3 a 5 têm complexidade $O(n^2)$. Pela regra da soma para o Big-O, $O(n) + O(n^2) = O(n^2)$. Portanto, o termo n^2 domina o custo final.

6) **Infeção de Rede (Ransomware)**:

Resposta: A cada novo nível, o número de servidores infectados dobra (uma função recursiva fazendo 2 chamadas para $n - 1$). A complexidade forma uma árvore binária de chamadas. Portanto, a complexidade Big-O é Exponencial: $\mathbf{O(2^n)}$, onde n é o nível de profundidade.

7) **Simplificação Assintótica**: Dada a função de custo $T(n) = 5n^2 + 100n + 500$, identifique a complexidade Big-O.

Resposta: Ao analisar a função, descartamos os termos de menor ordem ($100n$ e 500) e as constantes multiplicativas (5). O termo dominante é n^2, logo a complexidade é $\mathbf{O(n^2)}$.

8) **Busca Binária**:

Resposta: Complexidade: $\mathbf{O(\log_2 n)}$ (crescimento logarítmico).
Para $n = 1.000.000$, o número máximo de comparações é aproximadamente $\log_2(\mathbf{1.000.000}) \approx \mathbf{20}$ comparações.

9) O algoritmo abaixo compara pares adjacentes e os troca. Preencha a tabela de análise:

ORDENA(A, n)

	instrução	**custo**	**vezes** (Pior Caso)
1	**para** $i = 1$ **até** n:	c_1	$n+1$
2	**para** $j = 1$ **até** $n-1$:	c_2	$n(n)$
3	**se** $A[j] > A[j+1]$:	c_3	$n(n-1)$
4	Troca$(A[j], A[j+1])$	c_4	$n(n-1)$

Resposta: A presença de dois laços aninhados que iteram proporcionalmente a n resulta em complexidade quadrática: $\mathbf{O(n^2)}$.

10) **Comparação de Curvas**: O algoritmo A possui $100n$ e o algoritmo B possui n^2. Determine a partir de qual valor de n o Algoritmo A torna-se mais eficiente.

Resposta: Para encontrar o ponto de intersecção, igualamos as funções: $n^2 = 100n \Rightarrow n = 100$. A partir de $\mathbf{n > 100}$, a função linear $100n$ cresce menos que a quadrática n^2, tornando o Algoritmo A mais eficiente.

11) **Questão de Concurso (CESPE / CEBRASPE - 2025 - Polícia Federal)**:

( ) Certo

(**X**) **Errado**

Resposta: Justificativa: A afirmação está incorreta. O crescimento linear $O(n)$ é **mais lento** e mais eficiente do que o crescimento loglinear $O(n \log n)$.

12) **Questão de Concurso (IV - UFG - 2025 - Câmara de Morrinhos - GO - Analista de TI)**:

A) $O(1)$

B) $O(n)$

C) $O(n^2)$

D) $O(\log n)$

D) $O(n!)$

13) **Questão de Concurso (COPEVE-UFAL - 2023 - UFAL - Analista de TI)**:

A) $O(n)$

B) $O(n^2)$

B) $O(n^3)$

C) $O(\log n)$

D) $O(n \log n)$

Resposta: Justificativa: A função tem dois blocos principais. No caso verdadeiro (‘n $>$ 1000‘), existem dois laços ‘para‘ aninhados percorrendo até n, gerando custo $O(n^2)$. No pior caso para grandes entradas, a complexidade é $O(n^2)$.